

Python fundamental Mini Projects

Objective:

To develop a strong understanding of basic Python programming concepts by implementing different tasks using **variables, data types, conditional statements, loops, control statements, built-in functions, and user-defined functions**. The tasks aim to improve problem-solving and logical thinking skills through practical Python programs.

Tasks:

Data Cleaning: Extract and Format Employee Details

Scenario: You received raw HR data containing inconsistent formatting. You need to extract the employee name, email, and ID properly.

Data:

record = " EmployeeName : john doe "

Email = "JOHN.DOE@company.COM"

ID = " emp_0087 "

Task Instructions:

Use string methods and slicing to:

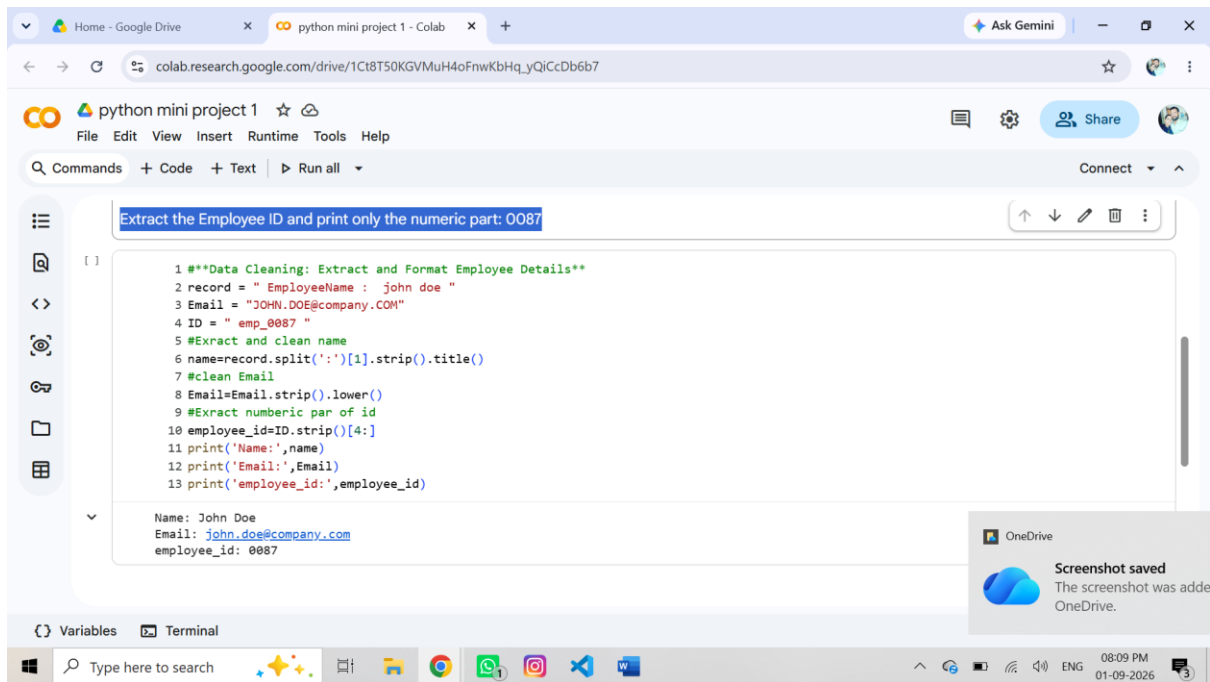
Remove unwanted spaces.

Convert:

Name → John Doe

Email → john.doe@company.com

Extract the Employee ID and print only the numeric part: 0087



Student Management & Academic Record System

🎯 Objective:

Understand how to use Tuples (immutable data) and Lists (mutable, structured data) together to manage and organize information efficiently.

🧩 Scenario:

- A school wants to maintain student data and marks for analysis.
- Each student's personal details are fixed (Name, Roll Number, Course) → stored in a Tuple.
- Each student's subject marks are modifiable → stored in a List.
- A collection of all students' marks is stored as a 2D List.
- For advanced organization, a 3D List is used to represent marks for multiple batches.

```
15 print("Marks:", marks2)
16
17 print("Student 3:", student3)
18 print("Marks:", marks3)
19
20 # Step 3: Combine all marks into a 2D List
21 batch1 = [marks1, marks2, marks3]
22
23 print("\nBatch 1 Marks:")
24 print(batch1)
```

Student 1: ('Kalai', 101, 'English')
Marks: [85, 90, 78]
Student 2: ('Kathir', 102, 'Maths')
Marks: [75, 88, 92]
Student 3: ('Lattu', 103, 'Tamil')
Marks: [80, 85, 89]

Batch 1 Marks:
[[85, 90, 78], [75, 88, 92], [80, 85, 89]]

Project Title: Retail Inventory Manager

Scenario: You are managing an inventory system for a retail shop. Certain product categories are fixed, but the actual products can be added or removed by the staff.

Tasks:

- Create a tuple with fixed product categories like Electronics, Stationery, and Clothing.
- Create a list of products that can be updated dynamically.
- Ask the user to enter a new product and add it to the list.
- Ask the user to input the index of a product to remove it from the list.
- Display the final list of products and the product categories.

```
[ ] 1 #Create a tuple with fixed product categories like Electronics, Stationery, and Clothing.
2 #fixed product categories
3 Categories=("Electronics", "Stationery", "Clothing")
4
5

[ ] 1 #Create a list of products that can be updated dynamically
2 Products=["Laptop", "Pen", "Shirt"]

[ ] 1 #Ask the user to enter a new product and add it to the list.
2 #add New product
3 new_product=input("Enter new product:")
4 Products.append(new_product)

Enter new product:Book

[ ] 1 #Ask the user to input the index of a product to remove it from the list.
2 index=int(input("enter index to remove:"))
3 Products.pop(index)
```

List Methods and Slicing

Project Title: Contact Directory

Scenario:

You have been assigned to build a contact management system where employees can store and retrieve contact details efficiently.

Tasks:

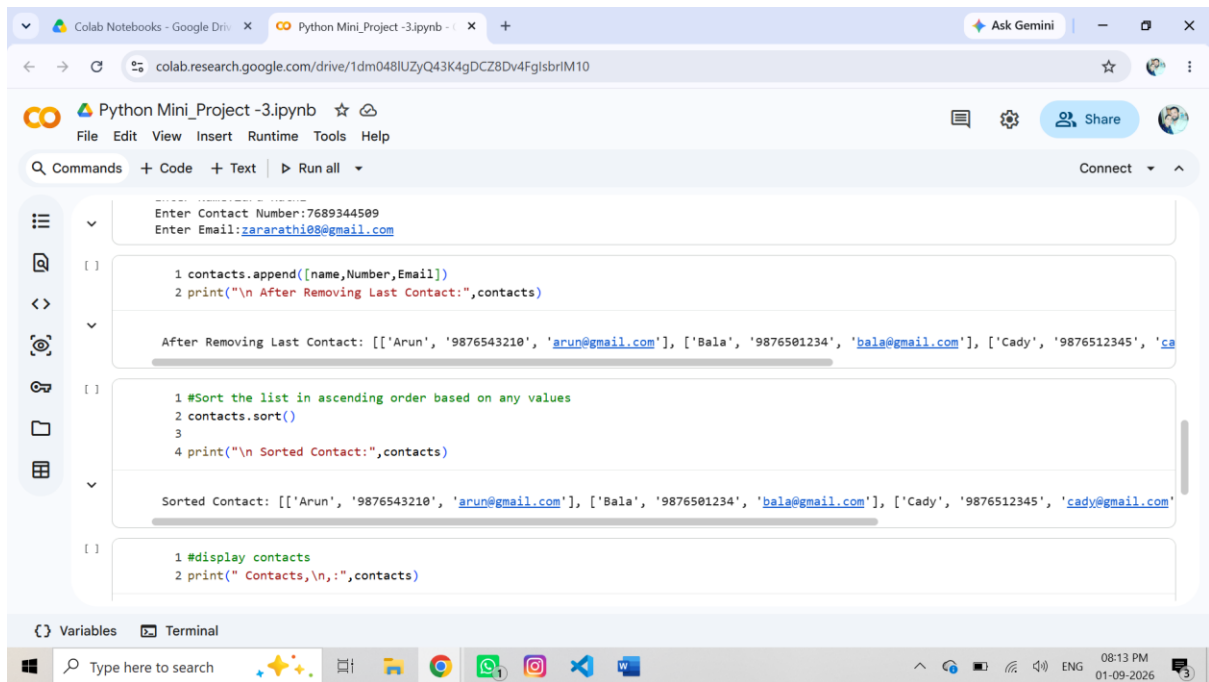
Create a 2D list where each row holds a contact's name, phone number, and email address.

Allow the user to add a new contact.

Remove the last item

Get new items from the users using input() and add those items to the list

Sort the list in ascending order based on any values



Topic: Departmental Data Tracker

Instructions:

Create a dictionary department with keys: "dept_name", "faculty", "courses", "budget".

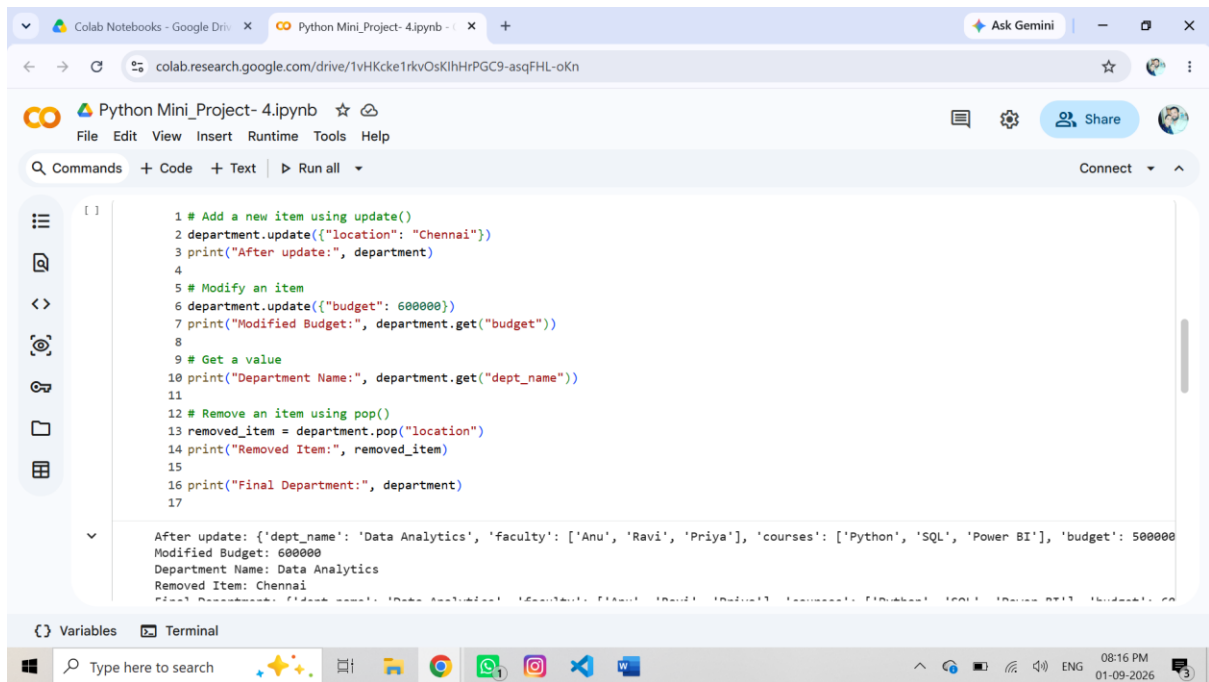
Add and modify items using dictionary methods (update, pop, get, etc.).

Create two sets: online_courses and offline_courses.

Perform union, intersection, difference, and symmetric_difference.

Create a frozenset named permanent_faculty.

Display all results without loops



```
1 # Add a new item using update()
2 department.update({"location": "Chennai"})
3 print("After update:", department)
4
5 # Modify an item
6 department.update({"budget": 600000})
7 print("Modified Budget:", department.get("budget"))
8
9 # Get a value
10 print("Department Name:", department.get("dept_name"))
11
12 # Remove an item using pop()
13 removed_item = department.pop("location")
14 print("Removed Item:", removed_item)
15
16 print("Final Department:", department)
17
```

After update: {'dept_name': 'Data Analytics', 'faculty': ['Anu', 'Ravi', 'Priya'], 'courses': ['Python', 'SQL', 'Power BI'], 'budget': 500000}

Modified Budget: 600000

Department Name: Data Analytics

Removed Item: Chennai

Final Department: {'dept_name': 'Data Analytics', 'faculty': ['Anu', 'Ravi', 'Priya'], 'courses': ['Python', 'SQL', 'Power BI'], 'budget': 600000}

Title: Scholarship Eligibility Checker

Instructions:

Create variables: studentname, marks, attendance, familyincome.

Use conditional statements (if, elif, else) to:

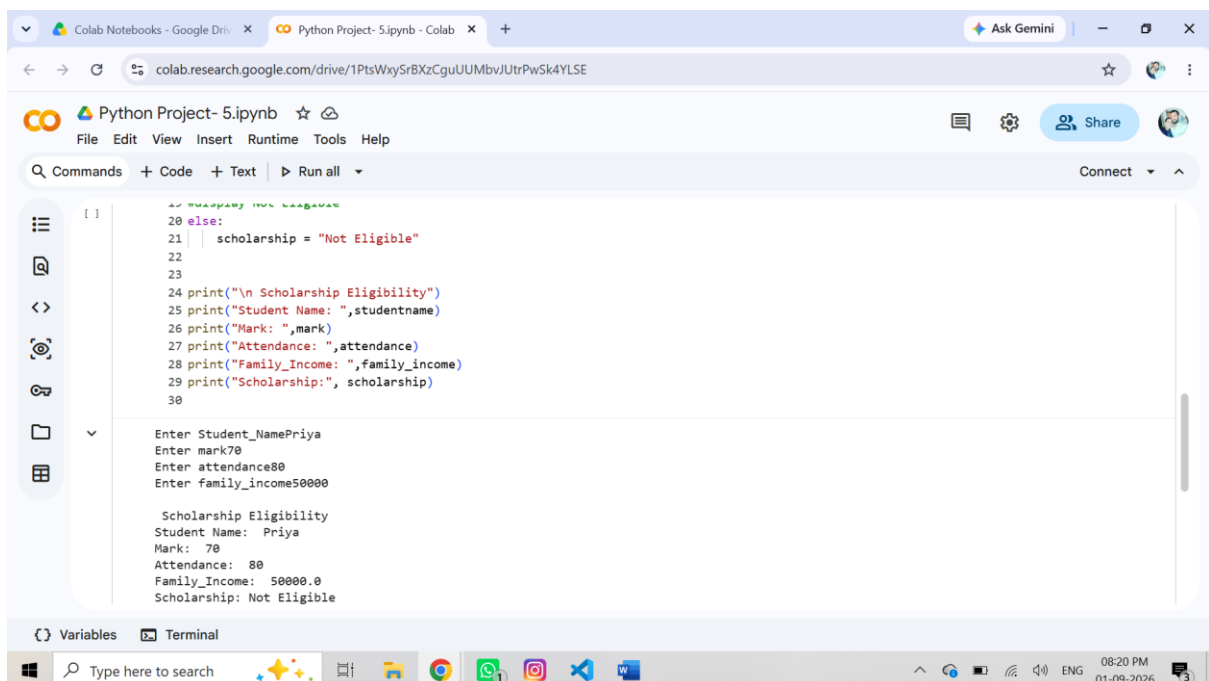
Award Full Scholarship if marks ≥ 90 and attendance ≥ 90 .

Award Half Scholarship if marks ≥ 80 or attendance ≥ 85 .

Award Need-Based Scholarship if marks ≥ 70 and family_income < 30000 .

Otherwise, display Not Eligible.

Display all data.



```
20 else:
21     scholarship = "Not Eligible"
22
23
24 print("\n Scholarship Eligibility")
25 print("Student Name: ", studentname)
26 print("Mark: ", mark)
27 print("Attendance: ", attendance)
28 print("Family_Income: ", family_income)
29 print("Scholarship: ", scholarship)
30
```

Enter Student_NamePriya

Enter mark70

Enter attendance80

Enter family_income50000

Scholarship Eligibility

Student Name: Priya

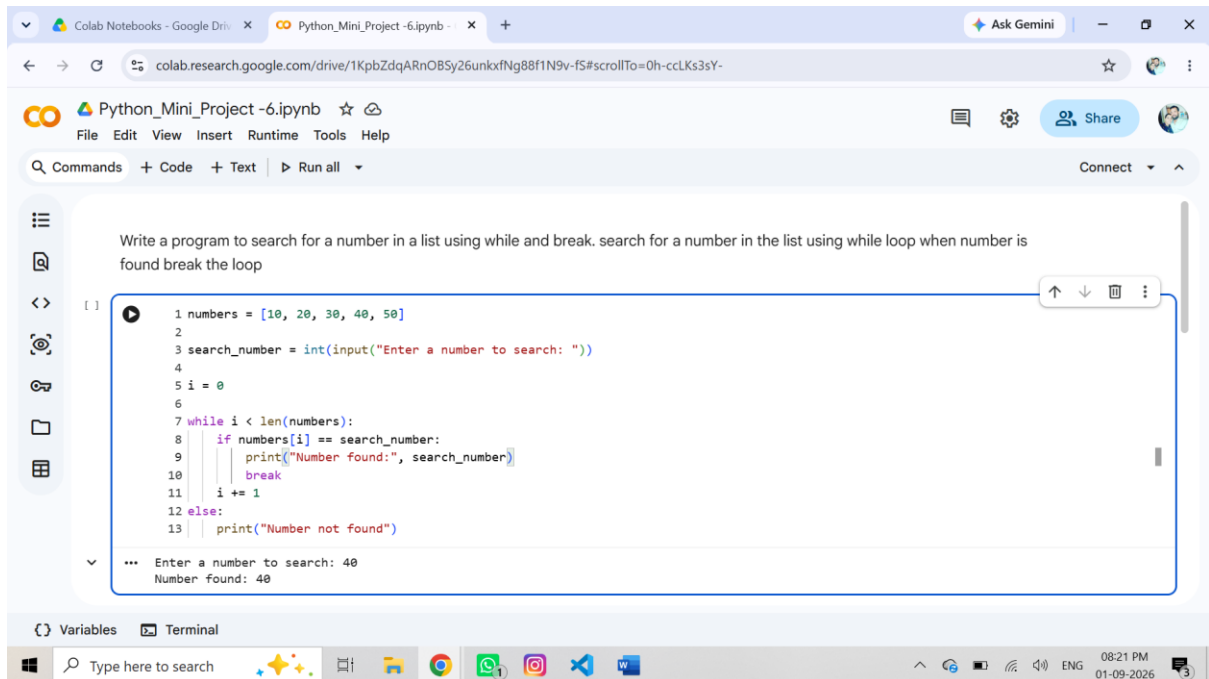
Mark: 70

Attendance: 80

Family_Income: 50000.0

Scholarship: Not Eligible

Write a program to search for a number in a list using while and break. search for a number in the list using while loop when number is found break the loop



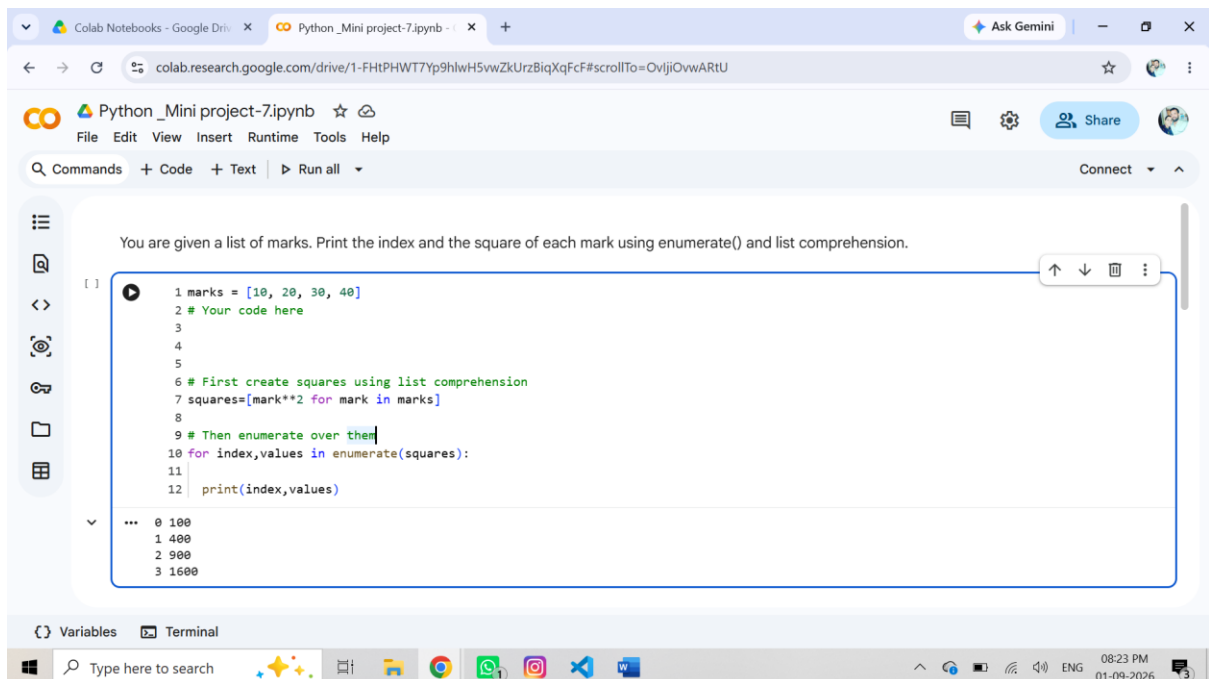
The screenshot shows a Google Colab notebook titled "Python_Mini_Project -6.ipynb". The code cell contains the following Python code:

```
1 numbers = [10, 20, 30, 40, 50]
2
3 search_number = int(input("Enter a number to search: "))
4
5 i = 0
6
7 while i < len(numbers):
8     if numbers[i] == search_number:
9         print("Number found:", search_number)
10        break
11    i += 1
12 else:
13     print("Number not found")
```

The output of the code is:

```
... Enter a number to search: 40
Number found: 40
```

You are given a list of marks. Print the index and the square of each mark using enumerate() and list comprehension.



The screenshot shows a Google Colab notebook titled "Python_Mini project-7.ipynb". The code cell contains the following Python code:

```
1 marks = [10, 20, 30, 40]
2 # Your code here
3
4
5
6 # First create squares using list comprehension
7 squares=[mark**2 for mark in marks]
8
9 # Then enumerate over them
10 for index,values in enumerate(squares):
11
12     print(index,values)
```

The output of the code is:

```
... 0 100
1 400
2 900
3 1600
```

Challenge Time Here's a nice for-loop-exercise:

Create Pattern Printing (Square) Program :

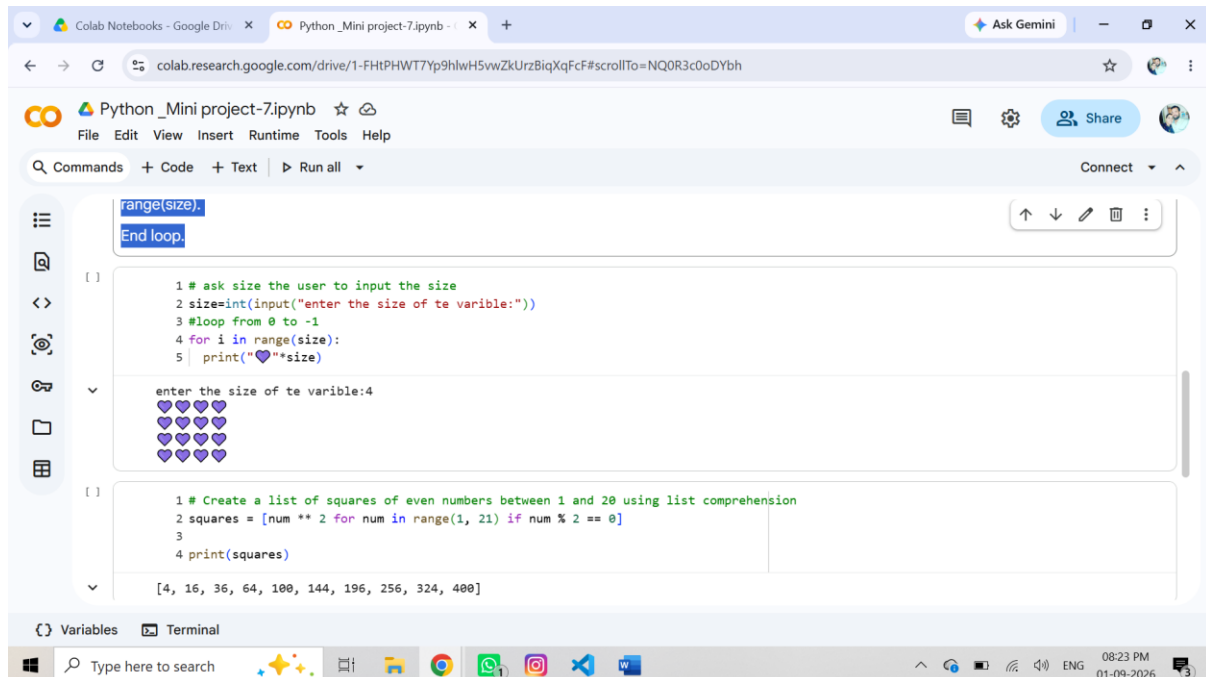
Algorithm:

Ask the user to input the size of the square. Read and store the user input size in a variable called size. Start a loop to iterate over the range of values from 0 to size-1 (inclusive). Let the loop variable be i.

Inside the loop:

Print size asterisks () horizontally. To achieve this, use the print() function with "" multiplied by size. Repeat step for each value of i in the range(size).

End loop.



The screenshot shows a Google Colab notebook titled "Python_Mini project-7.ipynb". The code is as follows:

```
range(size).  
End loop.  
  
[ ]  
1 # ask size the user to input the size  
2 size=int(input("enter the size of te variable:"))  
3 #loop from 0 to -1  
4 for i in range(size):  
5 | print("♥"*size)  
  
enter the size of te variable:4  
♥♥♥♥  
♥♥♥♥  
♥♥♥♥  
♥♥♥♥  
  
[ ]  
1 # Create a list of squares of even numbers between 1 and 20 using list comprehension  
2 squares = [num ** 2 for num in range(1, 21) if num % 2 == 0]  
3  
4 print(squares)  
  
[4, 16, 36, 64, 100, 144, 196, 256, 324, 400]
```

Built-in Functions and User-Defined Functions

Project Title: Simple Calculator App

Scenario: A company wants to provide users with a simple calculator to perform basic mathematical operations.

Tasks:

Create functions for addition, subtraction, multiplication, and division.

Ask the user to input two numbers and the operation they want to perform.

Call the appropriate function and display the result.

Use built-in functions like abs(), round(), and pow() within your solution. Handle division by zero gracefully and inform the user.

Colab Notebooks - Google Drive Python_Mini project-8.ipynb Ask Gemini

colab.research.google.com/drive/1u2HD3p9E_5H2a7-KFnEd82DmgqKXmbCH#scrollTo=iM50uTtUJV56

Python_Mini project-8.ipynb File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Connect

```
1 # Addition function
2 def add(a,b):
3     return a+b
4
5
6 # Subtraction function
7 def subtract(a,b):
8     return a-b
9
10
11 # Multiplication function
12 def multiple(a,b):
13     return a*b
14
15
16
17 # Division function
18 def devision(a,b):
19     if b==0:
20         return "cannot divide by zero"
21     return a/b
```

Variables Terminal

Type here to search

08:25 PM 01-09-2026

Colab Notebooks - Google Drive Python_Mini project-8.ipynb Ask Gemini

colab.research.google.com/drive/1u2HD3p9E_5H2a7-KFnEd82DmgqKXmbCH#scrollTo=iM50uTtUJV56

Python_Mini project-8.ipynb File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

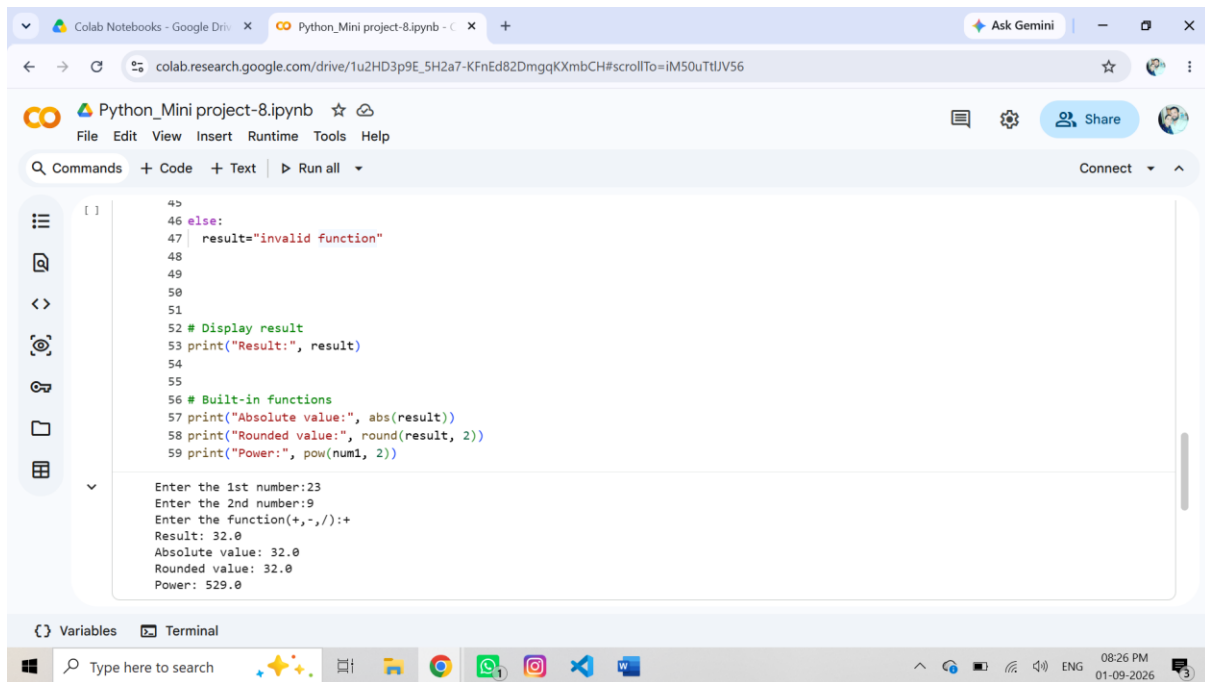
Connect

```
25
26
27
28 # Get input from user
29 num1=float(input("Enter the 1st number:"))
30 num2=float(input("Enter the 2nd number:"))
31 choose_function=input("Enter the function(+,-,/):")
32
33 #call Appropriate fountion
34 if choose_function=="+":
35     result=add(num1,num2)
36
37 elif choose_function=="-":
38     result=subtract(num1,num2)
39
40 elif choose_function=="*":
41     result=multiple(num1,num2)
42
43 elif choose_function=="/":
44     result=devision(num1,num2)
45
46 else:
```

Variables Terminal

Type here to search

08:25 PM 01-09-2026



```
45
46 else:
47     result="invalid function"
48
49
50
51
52 # Display result
53 print("Result:", result)
54
55
56 # Built-in functions
57 print("Absolute value:", abs(result))
58 print("Rounded value:", round(result, 2))
59 print("Power:", pow(num1, 2))
```

Enter the 1st number:23
Enter the 2nd number:9
Enter the function(+,-,/):+
Result: 32.0
Absolute value: 32.0
Rounded value: 32.0
Power: 529.0

Analysis:

The overall tasks provided practical experience in writing and executing Python programs. Different concepts such as **while loops, if-else conditions, break, continue, random number generation, built-in functions, and user-defined functions** were applied to solve real-world programming problems. Functions were used to make programs reusable and organized. Built-in functions such as `abs()`, `round()`, `pow()`, and `isinstance()` were also explored. These tasks helped in understanding how Python concepts work together to create simple and efficient programs.

Conclusion:

The overall tasks helped to build a strong foundation in **Python programming and problem-solving**. By completing these practical exercises, I gained a better understanding of loops, conditions, functions, control statements, and built-in functions. These concepts provide a good foundation for developing more advanced Python applications and progressing toward **Data Analytics and programming projects**.